Elvex – Documentation

Lionel Clément

Version 2.28.0

Contents

Introduction **2**

1 Full Example: Getting Started **2**
1.1 Syntagmatic description (*Context-Free Grammar rules*) . . . 3
1.2 Operational semantics . . . 3

2 Elvex notations **8**
2.1 General syntax . . . 8
2.2 Operational Semantics . . . 9
2.3 Macros . . . 19
2.4 Lexicon . . . 20
2.5 Input . . . 23

3 Grammar design patterns **25**

4 The `elvex` command **29**
4.1 `Elvex` command . . . 29
4.2 Grammar debugging . . . 32
4.3 `elvexlexicon` for managing compacted lexicons . . . 33

Introduction

`Elvex` is a surface realisation system based on typed feature structures, lexical resources and syntagmatic rules. It takes as input a feature structure representing a concept or a generation request, combines it with a lexicon and a grammar, and produces one or more textual realizations.

The central idea of `Elvex` is to describe generation by means of rules whose right-hand side symbols exchange information through inherited and synthesized feature structures. Inherited structures, written $\downarrow_i$, transmit information from a rule to its components, while synthesized structures, written $\Downarrow_i$, return information produced by these components. This makes it possible to express agreement, selection, optional constituents, alternatives and lexical constraints in a uniform way.

This document introduces the main elements of the language: feature structures, lexicons, rules, operational semantics and command-line tools. It also describes mechanisms used during generation, such as optional symbols, alternatives, deferred evaluation, tracing and linearization constraints. The examples are intended to show both the concrete syntax and the style of grammar writing expected by `Elvex`.

The documentation is organized progressively. The first sections present the linguistic and formal notions used by the system. The following sections describe the syntax of grammars and lexicons, then the operational statements available inside rules. The final sections explain how to run `Elvex`, how to debug grammars, and how to build or consult compacted lexicons.

1 Full Example: Getting Started

An example will familiarize the reader with `Elvex` before we go any further. Let us take the case of the text generation of a nominal phrase in French.

Example

```
1 NP → det N {
2    ↓2 = ↑;
3    ↓1 = ⇓2;
4    ⇑ = ⇓2;
5 }
6
7 N → adj N {
```

```
 8    [mod:<$Head::$Tail>, $Rest];
 9    [number:$inhNb] ⊂ ↑;
10    ↓1 = [number:$inhNb, gender:$synthGd, $Head];
11    ↓2 = [mod:$Tail, $Rest];
12    [gender:$synthGd] ⊂ ⇓2;
13    ⇑ = ⇓2 ∪ [qual:yes];
14 }
15
16 N → n {
17    [mod:NIL];
18    ↓1 = ↑;
19    ⇑ = ⇓1;
20 }
```

1.1 Syntagmatic description (*Context-Free Grammar rules*)

The first rule of grammar (line 1) defines `NP` as consisting of `det` and `N` from left to right.

`det` is a word; it is defined in the lexicon that we will see later.

Lines 7) and 16) follow the same logic and we have the context-free grammar:

```
NP → det N;
N → adj N;
N → n;
```

The only sequence we can generate with this simple example is a noun, preceded by a determiner and possibly one or several adjectives.

Syntagmatic rules are therefore made to describe the order of components of phrases, sentences and speeches.

1.2 Operational semantics

Each rule contains a set of operations. These operations describe the functional properties of the language, which may or may not depend on the order of the words.

The basics

We will explain this semantics through the example given to the beginning of this section before giving details.

- The lines 2) and 3) express the nature of the links that exist between `NP`, the name `N` and its determiner `det`; the main one being that agreement in gender and number occurs between nouns and their specifier.

  - Line 2) assigns the given nominal phrase properties to the noun. The attribute of the name is noted $\downarrow 2$ (this is the second term of the right part of the rule), and nominal phrase's is noted $\uparrow$.
  - Line 3) retrieves information from analyses of `N` to assign them to the determiner. The attribute of the determiner is noted $\downarrow 1$ (this is the first term of the right part of the rule), the attribute from the name `N` is noted $\Downarrow 2$.
  - Line 4) assigns the analyse of `N` to the synthesized attribute `NP`.

  With the two first lines, the noun will be chosen in the lexicon according to the concept to be expressed (attribute of the nominal phrase $\uparrow$). A female singular determiner will be chosen if the gender of the noun is feminine and the nominal phrase is singular.

  This mechanism, which makes it possible to articulate lexical choices according to the concepts and syntactic properties according to lexical properties is at the heart of `Elvex` and implements two types of attributes: inherited attributes (noted $\uparrow$, $\downarrow_i$) and synthesized attributes (noted $\Uparrow$, $\Downarrow_i$).

  Inherited attributes are calculated from left to right of the syntagmatic rules. Synthesized attributes are calculated from right to left. To put it simply, we can say that the inherited attributes convey information from the concept to the lexicon, and that the synthesized attributes convey information from the lexicon to higher layers of the syntactic analysis. Thus inherited attributes allow you to propagate the input to the lexicon to make a lexical choice, whereas conversely, synthesized attributes allow to retrieve the lexical information of the chosen name according to the concept expressed and to propagate this information (grammatical features, selection restrictions, phraseological units constraints, etc.) to the entire nominal phrase.

- The lines 8) to 13) allow you to set the gender and number agreement of the noun and adjective and also to propagate the concepts that produce adjectives. We notice that in French, the number is a semantism expressed directly as a sign associated with the noun, whereas gender

is a grammatical feature given by the lexicon. The first will be propagated from the concept to the noun, the second will be propagated from the lexicon to the layers regulating the agreement.

- Line 8) is a *guard*, *i. e.* a condition to realize `N` → `adj N`. It expresses that the realization of an adjective is conditioned by the existence of a list `<$Head::$Tail>` as the value of an attribute `mod` in the inherited attribute ↑.

  `$Head` and `$Tail` are two variables that respectively define the head and the tail of a list, *i. e.* respectively the first item in the list and the rest of the list.

  if the condition is verified, `$Head`, `$Tail` and `$Rest` will be affected by values given by the subsumption[1] values of `[mod:<$Head::$Tail>, $Rest]` by ↑.
- Line 9) assign to the variable `$inhNb` (*Inherited noun*) the value of the `number` feature of the inherited attribute ↑. This allows you to retrieve the number given to the nominal phrase.
- Line 10) assigns to the inherited attribute of the adjective the head of the list `$Head` unified[2] with a structure that gives the synthesized gender `$synthGd` (*synthesized gender*) and the inherited number `$inhNb`.
- Line 11) assigns to the inherited attribute of the noun all values passed by inheritance (`$Rest`) and adds the feature `mod` with the value `$Tail`, which is the tail of the list that brings together all other modifiers of the noun (other adjective or a proposal for example).
- Line 12) allows you to assign the variable `$synthGd` which is the gender synthesized from the noun.
- Finally line 13) assigns to the synthesized attribute of the phrase the synthesized attribute of the noun and adds a feature `qual` which marks that this noun phrase is qualified (which makes it possible to restrict others modifications).

- The rules on lines 17) to 19) are not specific, except for the custody of rule 17) which implements the value `NIL`.

[1] A feature structure `A` subsumes a feature structure `B` iif all the features of `B` are present in `A` with the same values or with values that subsume those of the values of `A`.

[2] The unification of a feature structure `A` with a feature structure `B`, is the smallest feature structure `C` such that `A` subsumes `C` and `B` subsumes `C`.

– Line 17) is a guard which allows us to prevent the inherited attribute ↑ to contain any feature `mod`. `mod:NIL` means that the attribute `mod` does not exist in ↑.

– Lines 18) and 19) do not require any further comment.

It becomes clear by examining this simple example that the order of operational rules is not the one given by grammar, but the one dictated by the availability of operands. Here, a possible order of application of the rules is 8), 9), 11), 12), 10), and 13).

In the current version of `Elvex`, this order is calculated from the availability of operands during rule evaluation.

The advice I can give to develop a grammar is to write the rules as the synthesized attributes are available.

We see that the operational statements of syntagmatic rules are not evaluated strictly from left to right or from top to bottom, but according to the availability of the attributes they depend on.

Before going any further, let us give definitions on the ratings that allow us to understand the syntax of the rules.

Empty rules, optional terms

It is possible to write empty rules, *i.e.* components that have no realization in text production. It is also possible to put optional elements written between parentheses `()`.

Example: The following rule gives all possible combinations for clitic pronouns placed before the verb.

```
CLITICS → (clr) (cld) (cla) (cld) (clg) (cll);
```

In French, the dative clitic (`cld`) is sometimes placed before, sometimes placed after the accusative clitic (`cla`) according to the person of the pronoun.

(1) a. *Jean* ***me*** ***le*** *donne*
Jean **to_me**(dative) **it**(accusative) gives
Jean gives it to me

b. *Jean* ***le*** ***lui*** *donne*
Jean **it**(accusative) **to_him**(dative) gives
Jean gives it to him.

According to this rule, the phrase `CLITICS` can also be empty, due to the fact that each term is optional. A rule whose right-hand side contains only optional terms may therefore generate the empty string ϵ. Operationally, an absent optional component is represented by an empty forest whose span is $[i, i]$: no lexical form is consumed, but the component is nevertheless considered resolved. This is important because the rest of the rule may still refer to the presence or absence of this component with the presence tests described below.

For a rule with n independent optional components, the generator may in principle produce up to 2^n combinations. One of these combinations may be empty if all optional components are absent. If the output device prints one realization per line, this empty realization may appear as a blank line or may not be visually obvious.

Paradigms

Each element of a syntagmatic rule can be a list of terms separated by the character `|` instead of being a single term. In this case, the product phrase is constructed with one of the terms in the list.

Example: The following rule allows you to build a sentence without a subject, with a nominative clitic subject `cln` (*je*, *tu*, *il*, ...), or with a subject as a nominal group.

```
S → ( cln | NP) VP;
```

A grammar written by using massively rules with alternatives and optional terms is equivalent to a very large grammar. For example, the rule describing the order of clitic pronouns is equivalent to a set of 64 rules:

```
CLITICS → ;
CLITICS → clr ;
CLITICS → cld ;
...
CLITICS → clr cld ;
CLITICS → clr cld cla ;
... ...
```

The ellipses in this listing stand for omitted productions, not literal Elvex tokens. Complete rules without an action block end with a semicolon.

However, `Elvex` never builds this set of rules explicitly and remains efficient with large grammars written with alternatives and optional terms.

The use of alternatives and optional terms is therefore recommended to describe the order of words, without regard to the combinatorial problem.

Operational resolution of optional terms and alternatives Optional terms and alternatives are resolved by the generator before the corresponding component is generated. An optional term opens two branches: one branch where the component is absent and one branch where it is present. When the component is absent, the generator records an empty forest with the same start and end position. When the component is present, the optional mark is removed and the component is generated normally.

An alternative such as `NP|cln` also opens one branch for each possible variant. If the term is both optional and alternative, for example `(NP|cln)`, the optional decision is made first: the component may be absent, or it may be present and then one of the alternatives is selected.

These decisions are internal to the generation algorithm, but they are visible in the operational semantics through the presence tests `#i` and `#i . j`.

2 Elvex notations

2.1 General syntax

Elvex source files may contain comments and file inclusions.

Comments Two forms of comments are accepted:

- line comments, introduced by `//` and ending at the end of the line;
- block comments, delimited by `/*` and `*/`.

For example:

```
// This is a line comment

/*
   This is a block comment.
   It may span several lines.
*/
```

File inclusion An Elvex source file may include another file with the directive `#include` at the beginning of a line:

```
#include path/to/file.elvex
```

The included file is read as if its contents occurred at the position of the directive. This is useful for splitting large descriptions into several files.

When verbose mode is enabled, Elvex reports the opening of included files. The current lexer opens the supplied file name relative to the process working directory; an included path is not automatically resolved relative to the including file's directory.

A syntagmatic rule is written

$A \rightarrow B_1 B_2 \ldots B_k <$ *operational semantics* $>$

A rule with no operational statements ends in a semicolon; otherwise, it ends with a statement block in braces. The following symbols may occur in the right-hand side.

Where B_i is written:

- (B_i) if the symbol B_i is optional.
- $C_1|C_2|\ldots|C_n$ if B_i is a choice between the C_i

B_i, C_i can be lexical categories or symbols of the grammar.

2.2 Operational Semantics

Notations Let the rule $A \rightarrow B_1 B_2 \ldots B_i \ldots B_k$

- $\uparrow$: refers to the attribute inherited from A.
- $\downarrow_i$: refers to the attribute inherited from B_i.
- $\Uparrow$: refers to the synthesized attribute of A.
- $\Downarrow_i$: refers to the synthesized attribute of B_i.

ASCII alternatives Most symbolic operators also have ASCII alternatives. These alternatives are often more convenient when writing grammar files on a standard keyboard.

Symbolic notation	ASCII notation	Meaning
$\rightarrow$	`->`	rule arrow
$\cup$	`U`	unification
$\subset$	`<<<`	subsumption
$\neq$	`!=`	inequality
$\leq$	`<=`	less than or equal
$\geq$	`>=`	greater than or equal
$\vee$	`||`	logical disjunction
$\wedge$	`&&`	logical conjunction
$\Rightarrow$	`=>`	implication
$\Leftrightarrow$	`<=>`	equivalence
$\neg$	`!`	negation

For example, the following two rules use equivalent notations:

```
S → NP VP { ↓1 = ↑ ∪ [case:nom]; }

S -> NP VP { ↓1 = ↑ U [case:nom]; }
```

The named variables are noted by an identifier preceded by "$", the anonymous variables are noted "`_`", the constant values are noted by identifiers.

Although the lexer recognizes the words `TRUE` and `FALSE`, the supplied Bison grammar does not admit them as value or expression literals. Use supported constants or expressions instead.

The feature structures are noted $[f_1, f_2, \ldots, f_k]$ where f_i is a feature as

- a linked variable feature
- an anonymous variable feature
- `HEAD:<identifier of a lexeme>`, `HEAD:$X` or `HEAD:_` for a predicate, a predicate stored in a variable, or an anonymous predicate.
- `LEMMA:<identifier>`, `LEMMA:$X` for a lemma constraint.
- `FORM:"<form>"`, `FORM:$X` or `FORM:_` for a literal form, a form stored in a variable, or an anonymous form. Literal forms may be concatenated with `+`, for example `FORM:"New" + " " + "York"`.
- a pair $attr_i : val_i$
  - with $attr_i$ a named attribute

- and with val_i the value of an attribute that can be:
  - A named variable
  - An anonymous variable
  - A nil value (`NIL`)
  - An atomic constant
  - A set of atomic constants separated by `|`
  - A literal constant: integer or double
  - A feature structure
  - A list
  - An inherited or synthesized attribute: $\uparrow$, $\Downarrow_i$
  - A field access such as $\uparrow$`.attr` or $\Downarrow$`i.attr`

The lists are noted

- $< f_1, f_2, \ldots, f_k >$ with f_i as variables, constants, numbers, feature structures, lists, inherited or synthesized attributes, or field accesses.
- $<$ *head* :: *tail* $>$ with *head* as one list element and *tail* as a list. *head* refers to the first item on the list, *tail* to the rest of the list. The tail may also be `NIL`.

If one wishes to bring an operational semantics to the syntagmatic rule, one defines $<$ *operational semantics* $>$ which is a list of rules between braces `{...}`.

A rule may optionally be preceded by rule modifiers such as `@trace`, `@withSpaces`, `@withoutSpaces` or `@unordered`.

- `@trace` traces the application of the rule.
- `@withSpaces` generates spaces between the components of the rule.
- `@withoutSpaces` suppresses spaces between the components of the rule.
- `@unordered` allows the components of the rule to be generated in any order.

Each rule is written:

- {<list of rules >} A list of rules.

- `attest <expr>;` Explicit guard where `<expr>` is an boolean expression that must be checked.
- `[f_1, feat_2, \dots, f_k];` Guard where the feature structure must subsume $\uparrow$. The guard may contain the feature `HEAD` which represents the concept to be generated. Thus, a concept is not necessarily linked to a lexical entry with `Elvex`, unlike usual text generation systems; it can produce a syntactic constraint. Conversely, a syntactic constraint does not necessarily have to be linked to a specific concept.
- `print <expr_1>, ..., <expr_n>;` Displays one or several expressions at standard output (useful for debugging).
- `println <expr_1>, ..., <expr_n>;` Does the same and displays a new line.
- `eprint <expr_1>, ..., <expr_n>;` Displays one or several expressions on standard error (useful for debugging without changing generated output).
- `eprintln <expr_1>, ..., <expr_n>;` Does the same on standard error and displays a new line.
- `<expr_1> = <expr_2>;`

  An assignment of the expression $< expr_1 >$ with the value of the expression $< expr_2 >$. The expression $< expr_1 >$ is a complex variable, or simple variable, the expression $< expr_2 >$ is a constant or a complex variable or a simple variable such that there is a environment [3] of the syntagmatic rule to solve it.

  The assignments of $\uparrow$ and $\Downarrow_i$ are not possible, as these attributes are implicitly computed during text generation and modifying them would result in inconsistencies.

  The assignments are limited to these cases:

  - Assign a list
    - `<...> = <...>` Assigns a list to another one.
    - `<...> = $X` Assigned by the value of the variable `$X`

[3] An environment is an application ϕ from E to F such that E is a set of variables and F a set of constants or variables. The resolution of a variable α in an environment is $\beta = \phi^k(\alpha)$ such that β is a constant. It should be noted that `Elvex` also handles complex variables. In this case $\phi^k(\alpha)$ is replacing variables in terms by their constants.

– Assign an inherited attribute
  * ↓`i = $X` Assigned by the value of the variable `$X`
  * ↓`i = [...]` Assigned by a constant or variable feature structure.
  * ↓`i =` ↑ Assigned by an inherited attribute.
  * ↓`i =` ... ∪ ... Assigned by the result of the unification of two expressions (feature structures or attributes).
  * ↓`i =` ⇓`j` Assigned by a synthesized attribute.
  * ↓`i =` ↑`.attr` or ↓`i =` ⇓`j.attr` assigns the value obtained by field access.
– Assign a synthesized attribute
  * ⇑ `= $X`
  * ⇑ `= [...]`
  * ⇑ `=` ↑
  * ⇑ `=` ... ∪ ...
  * ⇑ `=` ⇓`j`
  * ⇑ `=` ↑`.attr` or ⇑ `=` ⇓`j.attr`
– Assign a simple variable
  * `$X = $Y`
  * `$X = a` Assigned by a constant or a literal.
  * `$X = <...>`
  * `$X = [...]`
  * `$X =` ↑
  * `$X =` ... ∪ ...
  * `$X =` ⇓`j`
  * `$X =` ↑`.attr` or `$X =` ⇓`j.attr` by field access
  * `$X = <expr>` by evaluating an arithmetical or logical expression
– Subsume a feature structure

  A feature structure is subsumed
  * `[...]` ⊂ ↑ by an inherited attribute.
  * `[...]` ⊂ ⇓`j` by a synthesized attribute
  * `[...]` ⊂ `$X` by the value of a variable.
  * `[...]` ⊂ ↑`.attr` or `[...]` ⊂ ⇓`j.attr` by field access.

– `if (<boolExpr>) <rules>`

  Interprets operational semantics `<rules>` if and only if the expression `<boolExpr>` is verified.

– `if (<boolExpr>) <rules_1> else <rules_2>`

  Interprets operational semantics `<rules_1>` if and only if `<boolExpr>` is verified, otherwise interpret `<rules_2>`.

– `deferred (<boolExpr>) <rules>`

  Delays the interpretation of `<rules>` until `<boolExpr>` can be evaluated. This is useful when a statement depends on synthesized attributes that are not immediately available.

– `order i < j < ...;`

  Adds a linearization constraint between the components of the right-hand side of the current syntagmatic rule. The indexes are one-based: `1` refers to the first component, `2` to the second one, and so on. For instance, `order 2 < 1;` states that the second component must be generated before the first one. The constraint is projected onto the components that are actually present: if an optional component is absent, the remaining components keep the relative order specified by the chain. The constraint is a generation and forest-ordering constraint, not a syntactic left-to-right parsing requirement: alternatives that cannot be generated are simply discarded, and the surviving alternatives may still satisfy the order constraint.

– `order << i;`

  Constrains component `i`, if present, to be generated before all other present components of the rule.

– `order >> i;`

  Constrains component `i`, if present, to be generated after all other present components of the rule.

– `order i, j, ... by ⇓.attribute;`

  Constrains the relative order of the listed right-hand-side components using the same synthesized feature path for each component. The indexes are one-based. The parser also accepts an inherited path beginning with ↓, and paths may contain multiple fields, such as `⇓.prosody.weight`. Each key must evaluate to a number. The generator evaluates the path separately on the selected components; it orders two components when their numeric

keys compare strictly. Equal keys do not impose a relative order (other constraints may do so). For example: `order 1, 2, 3 by ⇓.rank;`. The syntax `order by <...>;` is not supported.

A logical expression `<boolExpr>` is

- `<boolExpr>` ∨ `<boolExpr>` Or
- `<boolExpr>` ∧ `<boolExpr>` And
- ¬ `<boolExpr>` Negation
- `<boolExpr>` ⇒ `<boolExpr>` Material implication
- `<boolExpr>` ⇔ `<boolExpr>` Biconditional
- `<expr>` == `<expr>` Equality
- `<expr>` ≠ `<expr>` Difference
- `<expr>` Expression evaluated as Boolean: A feature structure is evaluated as true if it is not `NIL` or ⊥ (result of the failure of unification between two feature structures).

The expressions are:

- Integer evaluated as true if not zero
- Double evaluated as true if not zero
- String evaluated as true if not empty
- `<expr>` ∪ `<expr>` Unification of two feature structures
- ↑
- ⇑
- $\downarrow_i$
- $\Downarrow_i$
- ↑`.attr` and ⇓`i.attr` Field access
- `#i` and `#i . j` Presence tests for the generated components identified by one-based indexes
- `[...]` Feature structure
- `NIL` Null value of a feature
- `a|b...` Constant evaluated as true
- `$X` Variable
- `<...>` List evaluated as true if not null (`<>`)

Presence tests: `#i` and `#i . j` The expressions `#i` and `#i . j` test the presence of a component on the right-hand side of the current rule. Indexes are one-based.

- `#i` is true if the i-th component is present in the current branch. It is false if this component is an optional component that has been skipped.
- `#i . j` is true if the i-th component is present and its selected variant is the j-th alternative. It is false for the other variants and for an absent optional component.

For example, with the rule:

```
S → (NP | cln) VP { ... }
```

`#1` is true when either `NP` or `cln` is present, `#1.1` is true only when `NP` is selected, and `#1.2` is true only when `cln` is selected.

Presence tests may be temporarily undecidable while the generator has not yet resolved the corresponding optional term or alternative. In that case, a statement depending on the test is suspended and is tried again when the choice is known. This applies in particular to `if` statements:

```
S → (A) B {
  if (#1) {
    ↓2 = [x:A];
  }
  else {
    ↓2 = [x:noA];
  }
}
```

The `if` is not evaluated on the unresolved item `S → • (A) B`. It is evaluated only after the optional component has been resolved: the present branch makes `#1` true, and the absent branch makes it false. For a mandatory simple component, `#i` is true. For a mandatory alternative, `#i` is true once one variant has been selected, and `#i . j` identifies the selected variant.

Predefined string constants The lexer provides two predefined string constants that are useful for debugging:

- `__LINE__` expands to the current line number;
- `__FILE__` expands to the current file name.

They are treated as strings and may be used in expressions, for instance with printing statements:

```
eprintln("debug at " + __FILE__ + ":" + __LINE__);
```

Let's add algebraic expressions and numbers:

- `<expr>` $<$ `<expr>`
- `<expr>` $\leq$ `<expr>`
- `<expr>` $>$ `<expr>`
- `<expr>` $\geq$ `<expr>`
- `<expr>` $+$ `<expr>`
- `<expr>` $-$ `<expr>`
- `<expr>` $*$ `<expr>`
- `<expr>` % `<expr>`
- `<expr>` / `<expr>`
- $-$ `<expr>`
- `floor(<expr>)` Integer floor of a numeric expression
- `min(<expr>, <expr>)` Minimum of two numeric expressions
- `max(<expr>, <expr>)` Maximum of two numeric expressions
- `rand()` Random number generator
- Double-precision floating-point format number. The current lexer accepts ordinary decimal notation, for example `1.25`, `-0.5` or `1e3`. Do not prefix a number with `d:` `d1.25` is not a numeric literal.

Variable evaluation

Variables are evaluated locally at a syntagmatic rule. This defines the scope of a variable.

Here are different methods for solving variables:

- Direct assignment

```
1 S → S {
2    [vform:NIL];
3    [tense:$tense, aspect:$aspect, mode:$mode] ⊂ ↑;
4    if (!$tense)
5      $tense = present;
6 ...
7 }
```

Line 5 indicates that the value of the variable `$tense` is `present` by default (in the absence of another value previously given). The value of `$tense` is eventually given by the inherited attribute ↑ on line 3.

Note that the order of evaluation of the rules is indifferent and that it is not necessary to precede rule (3) in the order that they are listed.

- Guard

```
S → NP VP {
   [subject:$inheritedSubject, $Rest];
...
}
```

The value of the variable `$inheritedSubject` is given by the ↑ subject feature. The value of the variable `$Rest` is the feature structure that remains when you remove the feature `subject`.

- Subsumption

```
S → NP VP {
...
   [subject:$synthesizedSubject] ⊂ ⇓2;
...
}
```

The value of the variable `$synthesizedSubject` is given by the `subject` feature of ⇓ 2.

- Binding a variable within a syntagmatic rule

```
S → NP VP {
   [subject:$inheritedSubject, $Rest];
   [subject:$synthesizedSubject] ⊂ ⇓2;
```

```
  ↓1 = [subject:$inheritedSubject]
                    ∪ [subject:$synthesizedSubject];
...
}
```

The variables `$synthesizedSubject` and `$inheritedSubject` are both used within this rule to build the `subject` feature of the inherited attribute $\downarrow 1$.

- Linking a variable within a lexical entry

```
a        v[HEAD:brouillard, subjC:[FORM:"il"],
                    objC:[HEAD:brouillard, det:yes,
                    def:no, part:yes, neg:$Neg],
                    locCl:[HEAD:_pro, number:sg,
                              person:three],
                    vform:tensed, vtense:present,
                    mode:indicative,
                    subj:[person:three, number:sg],
                    neg:$Neg, fct:none];
```

The variable `$Neg` is linked in this lexical entry to the expression "it is foggy" so that its negative use propagates a negation to the noun:

(2) a. *Il y a du brouillard* (it is foggy)
b. *Il n'y a pas de brouillard* (There is no fog)
c. **Il y a de brouillard* ()
d. **Il n'y a pas du brouillard*

2.3 Macros

Macro files contain definitions of the form:

```
@name = [feature:value, ...];
```

A macro expands to the feature structure on the right-hand side. It can be used as an item in a feature structure, either alone or mixed with explicit features:

```
@commonNoun = [cat:noun, number:sg];
```

The following rule, stored in a separate rules file, refers to that macro:

```
N → n  {
   ↓1  =  [ @commonNoun ,  HEAD: $H ] ;
}
```

If a macro name is unknown, parsing fails with an error. Macro expansion is syntactic: the features stored in the macro are inserted where `@name` appears.

2.4 Lexicon

The lexicon is a list of entries separated by semicolons.

Each entry defines the sequence to be produced, followed by a disjunction (noted by the character `|`) of lexical categories optionally followed by a feature structure.

The sequence to be produced may be:

- A string of characters between `""`, in which case the sequence will be exactly this one. Note that the sequence may be empty.
- A string of characters between `""` containing variables. The sequence produced will be this one with the variables substituted by their respective values.
- `FORM`: the sequence produced will literally be the one given by the feature `FORM`; variables in the form are resolved during generation.
- `FORM` followed by an identifier in a lexical entry, for example `FORM title;`, declares an entry whose produced form is supplied dynamically.

The category is an identifier, namely the final symbol of grammar.

The feature structure allows to represent all the constraints imposed by the lexicon.

A few examples:

- `"?" interrogativeDot;`
- `"tag-$Line" tag[line:$Line];`
- `FORM title;`

- "elles-mêmes" itself [gender:fm, number:pl, person:three, itself:yes];

- serai aux_être [aux:être, voice:active, finite:yes,
  mode:indicative, vtense:future_anterieur, vform:tensed,
  subj:[person:one, number:sg]];

- // Lexical input "temperature" producing
  //"La température est de xxx degrés" (the temperature is xxx degrees Celsius"
  est v[HEAD:température,
  subjC:[HEAD:température, number:sg, det:yes, def:yes],
  pobjC:[HEAD:degré, number:pl, pcas:de, det:yes,
  detNum:[HEAD:num, value:$Deg]],
  vform:tensed, vtense:present, mode:indicative,
  subj:[person:three, number:sg], fct:none, value:$Deg]

  //"la température est douce" (the temperature is mild)
  |v[HEAD:température, subjC:[HEAD:température, number:sg,
  det:yes, def:yes], modC:[HEAD:doux, number:sg, gender:fm],
  vform:tensed, vtense:present, mode:indicative,
  subj:[person:three, number:sg], fct:few]

  //"la température est élevée" (the temperature is high)
  |v[HEAD:température, subjC:[HEAD:température, number:sg,
  det:yes, def:yes], modC:[HEAD:élevé, number:sg, gender:fm],
  vform:tensed, vtense:present, mode:indicative,
  subj:[person:three, number:sg], fct:much];

- // Lexical entry of "temperature".
  // producing "Il fait chaud" (It's hot)
  fait v[HEAD:température, subjC:[FORM:"il"],
  modC:[HEAD:chaud, number:sg, gender:ms], vform:tensed,
  vtense:present, mode:indicative,
  subj:[person:three, number:sg], neg:$Neg, fct:high]

  //"Il fait très chaud" (It's very hot)
  |v[HEAD:température, subjC:[FORM:"il"], modC:[HEAD:chaud,
  number:sg, gender:ms, mod:<[HEAD:très]>], vform:tensed,
  vtense:present, mode:indicative, subj:[person:three,

```
 number:sg], neg:$Neg, fct:very_high]

//"Il fait très froid" (It's very cold)
|v[HEAD:température, subjC:[FORM:"il"], modC:[HEAD:froid,
 number:sg, gender:ms, mod:<[HEAD:très]>], vform:tensed,
 vtense:present, mode:indicative, subj:[person:three,
 number:sg], neg:$Neg, fct:very_low]

//"Il fait froid" (It's cold)
|v[HEAD:température, subjC:[FORM:"il"], modC:[HEAD:froid,
 number:sg, gender:ms], vform:tensed, vtense:present,
 mode:indicative, subj:[person:three, number:sg],
 neg:$Neg, fct:low];
```

- ```
  // Lexical entry of "rain" producing
  // "Il y a une pluie fine" (A fine rain fell)
  a v[HEAD:pleuvoir, subjC:[FORM:"il"],
   objC:[HEAD:pluie, det:yes, number:sg, qual:yes,
    def:no, mod:<[HEAD:fin, pos:post]>],
   locCl:[HEAD:_pro, number:sg, person:three],
   vform:tensed, vtense:present, mode:indicative,
   subj:[person:three, number:sg], neg:$Neg, fct:few]

  // "il y a un peu de pluie" (It is raining a bit)
  |v[HEAD:pleuvoir, subjC:[FORM:"il"],
   objC:[HEAD:pluie, det:yes, number:sg,
   detForm:[FORM:"un peu de"]],
   locCl:[HEAD:_pro, number:sg, person:three],
   vform:tensed, vtense:present, mode:indicative,
   subj:[person:three, number:sg], neg:$Neg, fct:not_much]

  // "il y a beaucoup de pluie" (There is plenty of rainfall)
  |v[HEAD:pleuvoir, subjC:[FORM:"il"],
   objC:[HEAD:pluie, det:yes, number:sg,
   detForm:[FORM:"beaucoup de"]],
   locCl:[HEAD:_pro, number:sg, person:three],
   vform:tensed, vtense:present, mode:indicative,
   subj:[person:three, number:sg], neg:$Neg, fct:much]
  ```
  ```

```
// "il y a de fortes précipitations" (It is raining so much)
|v[HEAD:pleuvoir, subjC:[FORM:"il"],
 objC:[HEAD:précipitation,
        det:yes, number:pl, qual:yes, def:no, mod:<[HEAD:fort,
        pos:pre]>], locCl:[HEAD:_pro, number:sg, person:three],
        vform:tensed, vtense:present, mode:indicative,
        subj:[person:three, number:sg], neg:$Neg, fct:a_lot];
```

2.5 Input

The input is summarized by the type of phrase or text you want to generate, followed by a feature structure containing the concept and the illocutionary structure to be expressed.

`HEAD` allows you to define the named concept to be generated. The special features `HEAD`, `LEMMA` and `FORM` can also be accessed by field access, for example ↑`.HEAD`, ↑`.LEMMA` or ⇓`1.FORM`. This is useful when a rule has to inspect the concept, lemma or literal form stored in an inherited or synthesized feature structure.

examples

- `sentence [HEAD:raining, neg:yes]`

  Simple entry for the concept of *raining* producing

  (3) a. *Il ne pleut pas* (It is not raining)
      b. *Il n'y a pas de pluie* (There is no rain)

- ```
  sentence
     [HEAD:to_ask,
      i:[HEAD:menuisier, id:3, number:sg, gender:ms, def:yes],
      iii:[HEAD:apprenti, id:6, def:yes, gender:ms, number:sg],
      ii:[HEAD:scier,
          i:[idref:6],
          ii:[HEAD:poutre, number:sg, def:yes]
          ]
     ]
  ```

  Complex concept where the features `id` and `idref` make it possible to constrain co-references.

  This entry produces
  ```

(4) *Le menuisier demande à l'apprenti de scier la poutre*
The carpenterasks the apprentice to saw the beam
The carpenter asks the apprentice to saw the beam

- `sentence`
  `[HEAD:raining,`
  `tense:future,`
  `modSType:time,`
  `modS:<[FORM:"lundi 18 mars 2019", type:time]>`
  `]`

  Concept producing an adverbial on the basis of the constant "lundi 18 mars 2019" (Monday, the 18th of March 2019). This entry produces:

  (5) *Il va pleuvoir le lundi 18 mars 2019.* (It will rain on Monday, March 18, 2019)

- `sentence`
  `[HEAD:cause,`
  `i:[HEAD:raining],`
  `ii:[HEAD:se_couvrir, i:[HEAD:you], modality:devoir]`
  `]`

  Input for the following generated sentences:

  (6) a. *Il pleut, tu dois te couvrir* (It is raining, you have to dress warmly)
  b. *Il pleut, par conséquent tu dois te couvrir* (It is raining, therefore you have to dress warmly)
  c. *Tu dois te couvrir s'il pleut* (You have to dress warmly if it is raining)
  d. *Tu dois te couvrir car il pleut* (You have to dress warmly because it is raining)
  e. *S'il pleuvait, tu devrais te couvrir* (If it rained, you should dress warmly)
  f. *S'il pleut, tu devras te couvrir* (If it is raining, you should dress warmly)
  g. *Il pleuvrait, tu devrais te couvrir* (It would rain, you should dress warmly)

- sentence
  [HEAD:initiative,
   i:[HEAD:Jean],
   objC:[lexFct:Magn, mod:<[HEAD:audacieux]>],
   lexFct:Oper1
  ]

  Input containing features to fix lexical functions. The production is

  (7) *Jean prend une initiative très audacieuse.* (Jean takes a very bold initiative)

- sentence
  [HEAD:initiative,
   i:[HEAD:Jean],
   objC:[lexFct:Magn],
   lexFct:Oper1
  ]

  Input for

  (8) *Jean prend une belle initiative.* (Jean takes a fine initiative)

- sentence
  [HEAD:to_break,
   i:[HEAD:Jean],
   ii:[HEAD:carafe, number:sg, def:yes,
      mod:<[HEAD:little],[HEAD:white]>],
   diathesis:passive, tense:past, neg:yes,
   modV:<[HEAD:roughly]>
  ]

  Input containing a negative passive feature. The production is

  (9) *La petite carafe blanche n'avait pas été cassée brutalement par Jean.* (The small white carafe had not been broken brutally by Jean.)

3 Grammar design patterns

A design pattern applies in a general case which can be summarized by some constraints. It is not associated with a specific event, but to a recurring

organization of the grammar for which a single and coherent answer must be provided.

A - Local propagation of synthesized attributes

Context: A feature `A` that must constrain a result `R`, where `R` depends only on the lexicon or grammar.

Examples of applications:

- Production of a constrained noun in an idiomatic phrase (*breaking the **ice***)
- Conjugation to the passive voice

The general rule is written as follows

```
1 X → Y {
2   [A:$v, $rest];
3   [ARealization:R] ⊂ ⇓1;
4   ↓1 = [R:$v, $rest];
5 }
```

Example 1: The sentence *We basically got taken to the cleaners on that deal.* introduces the fixed noun *cleaners* which comes from the verb *take one to the cleaner*. Here the agent of the passive sentence is not represented.

```
1  VN → V (PP) PP {
2    [obj:NIL, obl:$Obl];
3    [subj:$subj] ⊂ ↑;
4    ↓1 = ↑;
5    ↓3 = $Obl;
6    [oblC:$oblSynt] ⊂ ⇓1;
7    if (#2) {
8      attest $oblSynt;
9      ↓2 = $oblSynt;
10   }
11   else {
12     attest ¬$oblSynt;
13   }
14 }
15
16 taken verb[HEAD:take_one_to_the_cleaners,
17          constI:[HEAD:cleaner, number:pl, def:yes]];
```

The `VN` production belongs in a rules file, whereas the `taken verb[...];` entry belongs in a lexicon file. They are shown together here only to illustrate their interaction.

Example 2: The agent of a clause is carried out by the grammatical function subject for conjugation to the active voice, and by the oblique function in *by* for a passive voice conjugation.

```
1  // agent as subject
2  Sentence → Sentence {
3    [agent:$agent, $rest];
4    [agentRealization:subject] ⊂ ⇓1;
5    ↓1 = [subject:$agent, $rest];
6    ⇑ = ⇓1;
7  }
8
9  // agent as oblique
10 Sentence → Sentence {
11   [agent:$agent, $rest];
12   [agentRealization:oblBy] ⊂ ⇓1;
13   ↓1 = [byObj:$agent, $rest];
14   ⇑ = ⇓1;
15 }
16
17 // patient as object
18 Sentence → Sentence {
19   [patient:$patient, agent:NIL, $rest];
20   [patientRealization:object] ⊂ ⇓1;
21   ↓1 = [object:$patient, $rest];
22   ⇑ = ⇓1;
23 }
24
25 // patient as subject
26 Sentence → Sentence {
27   [patient:$patient, agent:NIL, $rest];
28   [patientRealization:subject] ⊂ ⇓1;
29   ↓1 = [subject:$patient, $rest];
30   ⇑ = ⇓1;
31 }
32
33 // passive voice
```

```
Sentence → NP VERB PP {
   [subject:$subject, parObl:$parObl, $rest];
   ↓1 = $subject;
   ↓2 = $rest ∪ [voice:passive];
   ↓3 = $parObl;
   ⇑ = [agentRealization:oblBy,
          patientRealization:subject];
}

// active voice
Sentence → NP VERB NP {
   [subject:$subject, object:$object, $rest];
   ↓1 = $subject;
   ↓2 = $rest ∪ [voice:active];
   ↓3 = $object;
   ⇑ = [agentRealization:subject,
          patientRealization:object];
}
```

B - Transformation of the attribute inherited by a syntactic head

Context: An inherited attribute `A` that must constrain a realization `R` by a new predicate in generation.

Examples of applications:

- Use of a verbal tense auxiliary or a modal auxiliary
- Compound conjugation in French

The general rule is written as follows

```
X → Y {
   [A:K];
   [HEAD:$HEAD, A:k, $Rest] ⊂ ↑;
   ↓1 = [HEAD:L, arg:[HEAD:$HEAD, $Rest]];
}
```

Example: In French, the aspect is reflected in conjugated forms, but also in compound verb constructions (*être en train de* + infinitive, to be in the process of doing sth).

```
sentence → sentence {
  [aspect:progressive];
  [HEAD:$HEAD, aspect:progressive, i:$I, $Rest] ⊂ ↑;
  ↓1 = [HEAD:être_en_train_de,
        i:[HEAD:$HEAD, i:$I, $Rest]];
  ⇑ = ⇓1;
}
```

4 The `elvex` command

`Elvex` is a Natural Language Generator. It takes as input a concept, a local lexicon, a compacted lexicon, and a grammar, and then outputs a text corresponding to the concept written in natural language.

Compilation and full installation of `Elvex`

ELVEX 2.28.0 uses the top-level CMake configuration and an out-of-source build directory. It requires CMake 3.12 or later, a C++11 compiler, Flex, Bison, and the libxml2 development headers and library. For platform-specific packages and other requirements, see the repository's `INSTALL.md`.

From the repository root, configure and build the executable files with:

```
cmake -S . -B build
cmake --build build --parallel 8
ctest --test-dir build --output-on-failure
./build/bin/elvex --version
```

The executables are written to `build/bin/`. Installation into a system directory is optional, not required to run ELVEX from its source tree. If needed, configure `CMAKE_INSTALL_PREFIX` and use `cmake --install build`; elevated permissions are only necessary when the chosen installation directory requires them.

4.1 `Elvex` command

The `elvex` command is:

```
elvex [options] [<input>]*
```

The options are:

`-h|--help`

Displays help and exits

`-v|--version`

Displays the version number and exits

`-V|--verbose`

Enables verbose mode.

`-a|--reduce-all`

Reduces all rules during the generation process. This mode allows fine debugging and requires an understanding of the generation process to be mastered.

`-t|--trace`

Enables HTML tracing of rules marked `@trace`. For detailed tracing of all stages, use `--trace-all` or the individual `--trace-...` options.

`-r|--random`

Displays only one randomly selected output. Random choice may occur at several levels: lexical entry selection, forest node selection and final root forest selection.

`--seed <number>`

Sets the random seed for reproducible random generation. This option is useful with `--random` and also controls calls to `rand()` in operational statements.

`-f|--first`

Displays only the first output in the current deterministic generation order. Unlike `--random`, this option does not shuffle choices.

`--forest-only`

Builds the shared derivation forest without enumerating its output strings. This is useful when a grammar produces many realizations.

`--progress`

Sends periodic generation progress reports to standard error.

`--server-stdio` (also `--stdio-server`)

Reads one generation input per line from standard input. Each result is terminated by an ASCII record separator and output is flushed. This mode cannot be combined with `--input-file` or positional inputs.

`--strategy <exhaustive|sample|beam>`

Selects the derivation strategy. The default strategy is `exhaustive`.

`--max-rule-choices <integer>`

Defines the maximum number of rule or disjunction choices opened in `sample` mode.

`--beam-width <integer>`

Defines the maximum number of items kept per state in `beam` mode.

`--macros-file <file>`

File containing macro definitions. Each definition has the form `@name = [features];`.

`--rules-file <file>`

File containing the rules.

`--lexicon-file <file>`

File containing the local lexicon.

`--input-file <file>`

File containing the input.

`--compacted-lexicon-file <path/file>` or `--clf <path/file>`

Prefix of the compacted lexicon files. The files `<path/file>.tbl` and `<path/file>.fsa` are used.

`--xml <file>`

Writes an XML representation of the generation forest to `<file>`. This option is available only when `Elvex` has been compiled with XML output support.

`--max-length <integer>`

Defines the maximum length of the sequence to be produced.

`--max-usages <integer>`

Defines the maximum number of times the same rule can be used.

`--max-items <integer>`

Defines the maximum cardinality of the item sets.

`--max-time <integer>`

Defines the number of seconds before the program stops.

`--max-attempts <integer>`

Defines the maximum number of attempts for random generation.

When a grammar can generate the empty string, the corresponding realization contains no surface form. Depending on the output context, this may be shown as an empty line or may be visually invisible among the other realizations.

4.2 Grammar debugging

Grammar debugging is performed with the `elvex` command using trace options. For example:

```
cd data/examples/french-noun-phrases
../../../build/bin/elvex --trace \
  --rules-file french-noun-phrases.rules \
  --lexicon-file french-noun-phrases.lexicon \
  --input-file french-noun-phrases.input > trace.html
```

With tracing enabled, `elvex` produces an HTML document on standard output. To inspect the trace, redirect it to a file and open it in a web browser. The example above uses files shipped with the repository.

Where the specific options are:

`--trace-all`

Trace all generation steps

`--trace-init`

Trace the initial items.

`--trace-stage`

Trace the stabilized item set for each generation stage. A stage is printed once after normalization and closure; this is different from `--trace-close`, which traces the internal closure operations.

`--trace-close`

Trace the item set closure.

`--trace-shift`

Trace the items for each lexical entry.

`--trace-reduce`

Trace the items for each reduction.

`--trace-action`

Trace the operational semantics results.

4.3 `elvexlexicon` for managing compacted lexicons

The `elvexlexicon` command builds and inspects compacted lexicons. Compacted lexicons are intended for open lexical categories such as nouns, verbs, adjectives and adverbs. Closed or grammatical categories, such as prepositions, determiners, pronouns or complementizers, may remain in the local lexicon loaded directly by `elvex` with `--lexicon-file`.

The command is:

```
elvexlexicon [global options] <build|consult|list>
```

The action is mandatory:

`build`

Builds the compacted lexicon from a pattern file and a morphological file.

`list`

Lists the content of an existing compacted lexicon.

`consult`

Consults an existing compacted lexicon interactively.

The global options are:

`-h|--help`

Displays help and exits.

`-v|--version`

Displays the version number and exits.

`-V|--verbose`

Enables verbose mode.

`--compacted-lexicon-file <path/file>` or `--clf <path/file>`

Prefix of the compacted lexicon files. The files `<path/file>.tbl` and `<path/file>.fsa` are used.

`--macros-file <file>`

File containing macro definitions used in the pattern and morphological files. Each definition has the form `@name = [features];`.

`--pattern-file <file>`

Pattern file describing lexical entries at the lexeme level. It consists of lines of the form:

```
lexeme   pos   lemma   features
```

The `lexeme` is the lexical unit selected by the generation model. It may correspond to a simple word, a collocation or an idiomatic construction. The `lemma` identifies the morphological paradigm with which the lexeme is associated. All four columns must be separated by actual TAB characters, not spaces.

`--morpho-file <file>`

Morphological file describing inflected forms at the lemma level. It consists of lines of the form:

```
form   pos   lemma   features
```

The `form` is the produced surface form. The `lemma` is used to match a morphological entry with one or more pattern entries. This format also requires TAB separators between its four columns.

The compacted lexicon is built by combining pattern entries and morphological entries that share the same `pos` and `lemma`. Their feature structures are unified. This keeps a clear distinction between the lexeme, which represents a lexical choice in the generation model, and the lemma, which represents a morphological paradigm.

For example, a pattern file may contain adjective classes and their linearization rank:

```
LARGE    adj      large    [HEAD:large, type:size, rank:1]
PINK     adj      pink     [HEAD:pink, type:color, rank:4]
IRON     adj      iron     [HEAD:iron, type:material, rank:6]
```

and the morphological file may contain the corresponding surface forms:

```
large    adj      large    []
pink     adj      pink     []
iron     adj      iron     []
```

After compilation, the resulting entries can be used by grammar rules. For instance, the synthesized feature `rank` can later be exploited by a linearization instruction such as:

```
order 1, 2, 3 by ⇓.rank;
```

Command to build a compacted lexicon

```
elvexlexicon --clf <path/file> \
  --macros-file <macros-file> \
  --pattern-file <pattern-file> \
  --morpho-file <morpho-file> \
  build
```

The `--macros-file` option is needed only if the pattern or morphological files use macros.

Command to list a compacted lexicon

```
elvexlexicon --clf <path/file> list
```

Command to consult a compacted lexicon

```
elvexlexicon --clf <path/file> consult
```

The compacted lexicon is a set of lexical entries of the type

```
pos#lexeme form#FS|form#FS...
```

- `pos` is a part of speech
- `lexeme` is the lexeme given by the `HEAD` feature

- `form` is the produced form, represented as a string of UTF-8 characters
- `FS` is a feature structure